

QUERYPILOT LOCAL

Offline PostgreSQL Query Plan Assistant
with Microsoft Foundry Local

7 Günlük Hızlandırılmış Geliştirme, Test ve Teslim Raporu

Hazırlayan	Eray Kulkizaga
Program	Microsoft Türkiye AI Innovators Summer Program
Geliştirme dönemi	23-29 Temmuz 2026
Teslim tarihi	30 Temmuz 2026
Doküman sürümü	v1.0

Teslim hedefi: Çalışan yerel demo + GitHub repository + evaluation raporu + teknik sunum

İçindekiler

1. Yönetici özeti
2. Proje tanımı ve problem çerçevesi
3. Nihai MVP kapsamı ve başarı kriterleri
4. Kullanıcı akışı ve teknik mimari
5. Modül tasarımı ve güvenilirlik yaklaşımı
6. Veri, API ve çıktı sözleşmeleri
7. Repository ve geliştirme düzeni
8. Yedi günlük ayrıntılı uygulama planı
9. Test ve evaluation stratejisi
10. Güvenlik, etik kullanım ve sınırlamalar
11. Risk kaydı ve önleyici aksiyonlar
12. Sunum, demo ve teslim paketi
13. 30 Temmuz teslim kontrol listesi
14. CV ve portföy kullanımı
15. Kaynakça

Kapsam dondurma kararı

Bu raporda tanımlanan MVP 23 Temmuz itibarıyla dondurulmuştur. Teslim tarihine kadar yalnızca çalışan ana akışı, güvenlik kontrollerini, evaluation setini ve sunum kalitesini güçlendiren işler alınacaktır.

1. Yönetici özeti

QueryPilot Local; PostgreSQL sorgularının makine tarafından okunabilir execution planlarını analiz eden, ilgili performans bilgisini yerel bir knowledge base üzerinden bulan ve Microsoft Foundry Local ile kaynak gösteren açıklamalar üreten çevrimdışı bir AI yardımcı sistemidir.

Proje, tek başına bir dil modelinin SQL performansı hakkında tahmin yürütmesine dayanmaz. Teknik teşhis; EXPLAIN JSON çıktısı, deterministik kurallar ve ölçülebilir plan kanıtları üzerinden yapılır. Yerel RAG katmanı uygun PostgreSQL bilgisini getirir; Foundry Local ise bu bulguları doğrulanan bir rapora dönüştürür.

Yedi günlük teslim süresi nedeniyle ürün, kontrollü bir eğitim ve demo veritabanıyla sınırlıdır. Sistem yalnızca read-only SELECT sorgularını analiz eder; önerilen SQL hiçbir zaman otomatik uygulanmaz.

Tek cümlelik hedef

30 Temmuz 2026 tarihinde, bir SELECT sorgusunu güvenli biçimde analiz eden, EXPLAIN JSON planından teknik bulgular çıkaran, yerel kaynaklara dayalı öneri üreten ve en az 12 senaryoyla değerlendirilen çalışan bir v1.0 teslim etmek.

2. Proje tanımı ve problem çerçevesi

2.1 Çözülen problem

- PostgreSQL execution planları ağaç yapısında çok sayıda maliyet, satır tahmini, çalışma süresi ve buffer metriği içerir.
- Yeni geliştiriciler sequential scan, join stratejisi, disk tabanlı sort veya cardinality hatalarını yorumlamakta zorlanabilir.

- Genel amaçlı LLM cevapları gerçek execution planı ve kaynak metinlerle sınırlandırılmadığında güvenilir olmayabilir.
- Bulut tabanlı analiz, özel sorgu ve şema bilgilerinin dış servislere gönderilmesi açısından uygun olmayabilir.

2.2 Önerilen çözüm

QueryPilot Local; SQL güvenlik doğrulaması, PostgreSQL EXPLAIN çalıştırıcısı, recursive plan parser, deterministik rule engine, yerel embedding tabanlı retrieval ve Foundry Local rapor üretimini tek bir akışta birleştirir.

1. Kullanıcı hazır bir demo senaryosu seçer veya güvenli bir SELECT sorgusu girer.
2. Sorgu salt-okunur kurallara göre doğrulanır.
3. PostgreSQL üzerinde EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) çalıştırılır.
4. Plan ağacı recursive olarak parse edilir ve kanıtlar çıkarılır.
5. Rule engine, önceden tanımlanan problem kategorilerinden birini belirler.
6. Yerel RAG katmanı ilgili teknik kaynak parçalarını top-3 olarak getirir.
7. Foundry Local, bulgular ve kaynaklar üzerinden Pydantic ile doğrulanan rapor üretir.
8. Arayüz; teşhis, kanıt, öneri, kaynak ve latency bilgisini birlikte gösterir.

2.3 Portföy değeri

Proje; backend API geliştirme, PostgreSQL performans analizi, güvenli SQL yürütme, local AI inference, RAG, structured output, testing ve evaluation becerilerini aynı ürün akışında gösterir. SupportFlow AI müşteri destek RAG sisteminden farklı olarak database tooling ve query performance alanına odaklanır.

3. Nihai MVP kapsamı ve başarı kriterleri

3.1 P0 - teslim için zorunlu özellikler

Alan	P0 kapsamı
Yerel inference	Foundry Local üzerinden chat modeliyle rapor üretimi
Yerel embedding	Knowledge chunk'larının cihaz üzerinde vektörleştirilmesi
SQL güvenliği	Tek statement, yalnızca SELECT / WITH ... SELECT, yasaklı komutların reddi
Execution plan	EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) çıktısının alınması
Plan parser	Plan ağacının recursive okunması ve kanıtların normalize edilmesi
Rule engine	En az 4 teknik problem kategorisinin deterministik tespiti
Local RAG	Markdown knowledge base, top-3 cosine similarity retrieval
Structured output	Pydantic ile doğrulanan, citation içeren analiz raporu
Arayüz	Senaryo seçimi, SQL editörü, sonuç ve raw plan görünümü
Evaluation	En az 12 senaryo ve otomatik sonuç raporu
Kalite	Kritik unit/API testleri ve tek komutlu smoke test
Teslim	README, mimari diyagram, sunum PDF'i ve demo videosu

3.2 Problem kategorileri

Kategori	Başlangıç tespit yaklaşımı
potential_missing_index	Seçici filtreye rağmen geniş sequential scan veya yüksek Rows Removed by Filter
expensive_nested_loop	Yüksek loop sayısı ve pahalı inner plan ile nested loop davranışı
disk_based_sort	Sort Method bilgisinde external merge / disk kullanımı
cardinality_misestimation	Estimated rows ile actual rows arasında yüksek oran farkı
no_clear_issue	Kuralların güvenilir biçimde belirgin bir problem bulamaması

3.3 Kapsam dışı maddeler

- Production veritabanına bağlanma veya otomatik değişiklik uygulama
- INSERT, UPDATE, DELETE, MERGE, DDL veya çoklu statement analizi
- Multi-agent mimari, LangChain/LangGraph veya uzun süreli agent memory
- Authentication, multi-tenant SaaS, Redis/Celery ve görev kuyruğu
- pgvector, harici vector database veya cloud deployment
- Otomatik index oluşturma, query rewrite uygulama veya DBA yerine karar verme
- Gelişmiş React frontend ve mobil istemci

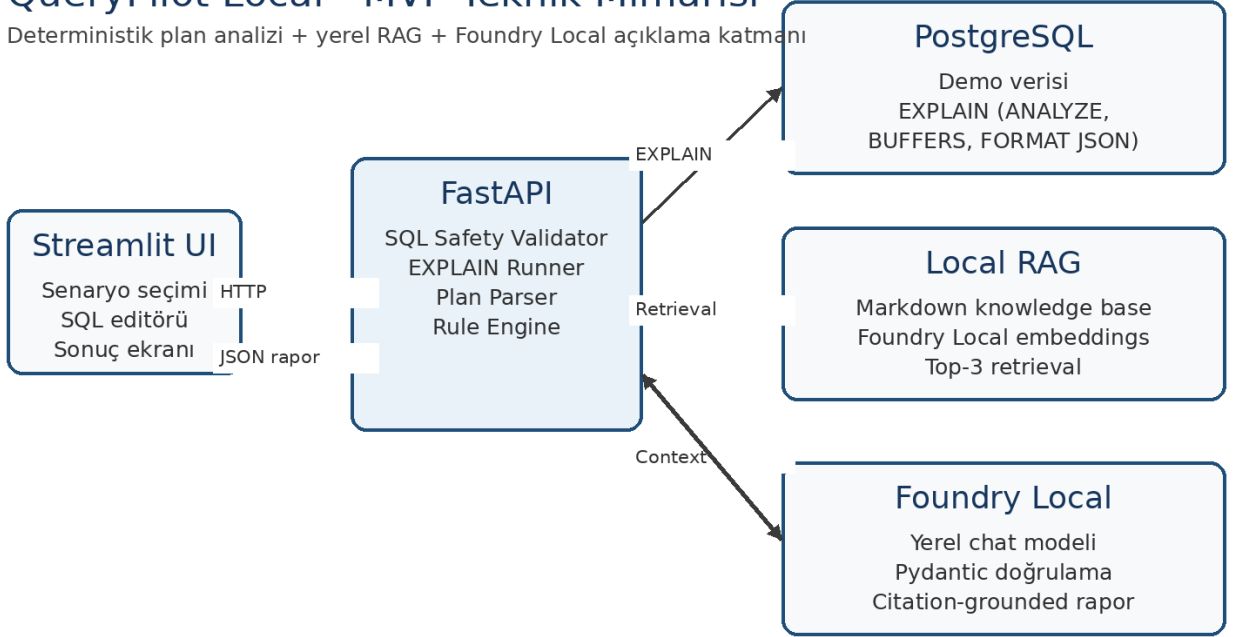
3.4 Başarı ölçütleri

Ölçüt	Kabul yaklaşımı
Uçtan uca akış	Dört canlı demo sorgusundan en az üçü hatasız analiz edilir
Rule diagnosis	12 senaryoda raporlanan doğruluk ölçülür ve README'de paylaşılır
Retrieval Hit@3	Beklenen kaynak ilk üç retrieval sonucu içinde aranır
Schema-valid rate	Pydantic doğrulamasını geçen çıktı oranı raporlanır
Citation-valid rate	Sadece retrieved set içindeki citation ID'lerine izin verilir
No-answer	Yetersiz kanıt senaryosunda kesin öneri üretilmez
Güvenlik	Yasaklı SQL testlerinin tamamı reddedilir
Tekrar üretilebilirlik	Temiz ortamda kurulum ve smoke test tamamlanır

4. Kullanıcı akışı ve teknik mimari

QueryPilot Local - MVP Teknik Mimarisi

Deterministik plan analizi + yerel RAG + Foundry Local açıklama katmanı



Şekil 1. QueryPilot Local MVP teknik mimarisi.

4.1 Ana servisler

Bileşen	Sorumluluk
Streamlit UI	Senaryo seçimi, SQL girişi, analiz tetikleme ve sonuç görüntüleme
FastAPI	Ana uygulama akışı, doğrulama, analiz, retrieval ve rapor endpointleri
PostgreSQL	Sentetik demo verisi ve execution plan üretimi
Local knowledge base	EXPLAIN, index, join, statistics ve sorting notları
Foundry Local	On-device embedding ve yerel açıklama üretimi
Evaluation runner	Fixture ve canlı senaryolar üzerinde otomatik ölçüm

4.2 Güvenilirlik ilkesi

Deterministik çekirdek, üretken açıklama

Rule engine teknik bulguyu belirler. RAG kaynak getirir. LLM; teşhisi icat etmek yerine, verilen plan kanıtını ve kaynakları anlaşılır bir rapora dönüştürür. LLM başarısız olduğunda sistem deterministik fallback raporu döndürür.

5. Modül tasarımı ve güvenilirlik yaklaşımı

Modül	Görev
sql_validator.py	AST tabanlı tek statement ve read-only kontrolü; yasaklı komutların reddi
explain_runner.py	Read-only kullanıcı, statement_timeout ve parametrik EXPLAIN yürütme
plan_parser.py	Plan node'larını recursive gezer; scan, join, sort, row ve timing verilerini normalize eder
rule_engine.py	Kural eşikleri üzerinden kategori, severity ve plan evidence üretir
chunker.py	Başlık farkındalıklı knowledge chunk üretimi ve metadata ataması
embedder.py	Foundry Local embedding adapter; batch üretim ve hata yönetimi
index.py / retriever.py	NumPy vektör deposu ve cosine similarity top-3 arama
foundry_client.py	Model alias, timeout ve tek provider adapter
generator.py	Prompt, structured output, citation integrity ve fallback raporu

5.1 SQL güvenlik doğrulaması

- Sadece tek SQL statement kabul edilir.
- Kök ifade SELECT veya WITH ... SELECT olmalıdır.
- INSERT, UPDATE, DELETE, MERGE, COPY, CALL, DO, CREATE, ALTER, DROP ve TRUNCATE reddedilir.
- Sorgu read-only PostgreSQL kullanıcısıyla çalıştırılır.
- statement_timeout başlangıçta 3 saniye olarak ayarlanır.
- EXPLAIN ANALYZE yalnızca sentetik yerel demo veritabanında çalıştırılır.

5.2 Başlangıç rule engine kuralları

Kural	MVP sinyali
Missing index sinyali	Seq Scan + yüksek Actual Rows veya Rows Removed by Filter + seçici filter ifadesi
Nested loop sinyali	Nested Loop + inner node loops yüksek + toplam süre belirgin
Disk sort sinyali	Sort Method metninde external merge veya Disk alanı
Cardinality sinyali	$\max(\text{actual_rows}, 1) / \max(\text{plan_rows}, 1)$ oranı ≥ 10 veya ≤ 0.1
Belirsiz sonuç	Hiçbir kural minimum kanıt eşiğini karşılamıyorsa no_clear_issue

5.3 RAG tasarımı

- Knowledge base: 6 kısa Markdown doküman.
- Chunk boyutu: yaklaşık 250-450 token; overlap yaklaşık 50 token.
- Embedding: Foundry Local cihaz içi embedding API'si.
- Depolama: vektörler .npz, metadata JSON.

- Retrieval: cosine similarity ve top-3 sonuç.
- Citation: document_id, title ve section alanları.

6. Veri, API ve çıktı sözleşmeleri

6.1 Analiz isteği

```
POST /api/v1/analyses
{
  "scenario_id": "missing_customer_email_index"
}
```

Alternatif olarak kullanıcı güvenli bir SQL sorgusu gönderebilir:

```
{
  "sql": "SELECT * FROM customers WHERE email = 'demo@example.com'"
}
```

6.2 Analiz cevabı

```
{
  "issue_category": "potential_missing_index",
  "severity": "high",
  "summary": "The query scans the full customers table.",
  "plan_evidence": [
    "Seq Scan on customers",
    "Rows Removed by Filter: 19999",
    "Execution Time: 18.6 ms"
  ],
  "recommendation": "Consider an index on customers.email.",
  "recommendation_sql": "CREATE INDEX idx_customers_email ON customers(email);",
  "citations": [
    {
      "document_id": "pg-indexes-01",
      "title": "PostgreSQL Indexes",
      "section": "Selective predicates"
    }
  ],
  "insufficient_context": false,
  "latency_ms": 1640
}
```

6.3 Pydantic çıktı modeli

```
class Citation(BaseModel):
    document_id: str
    title: str
    section: str

class AnalysisReport(BaseModel):
    issue_category: str
    severity: Literal["low", "medium", "high"]
    summary: str
    plan_evidence: list[str]
    recommendation: str
    recommendation_sql: str | None
    citations: list[Citation]
    insufficient_context: bool
```

6.4 Citation bütünlüğü

- Model yalnızca retrieval sonucunda sağlanan document_id değerlerini döndürebilir.
- Bilinmeyen citation ID çıktısı geçersiz kılar.
- Bir kez repair prompt çalıştırılır; ikinci hatada fallback raporu kullanılır.
- Kaynak bulunmadığında veya rule evidence zayıf olduğunda insufficient_context=true döner.

7. Repository ve geliştirme düzeni

```

querypilot-local/
├── app/
│   ├── main.py
│   ├── config.py
│   ├── schemas.py
│   ├── api/analyses.py
│   ├── database/{connection.py, explain_runner.py}
│   ├── analysis/{sql_validator.py, plan_parser.py, rule_engine.py}
│   ├── rag/{chunker.py, embedder.py, index.py, retriever.py}
│   └── llm/{foundry_client.py, generator.py, prompts.py}
├── ui/streamlit_app.py
├── knowledge/
├── evaluation/{scenarios.jsonl, run_evaluation.py, results.json}
├── sql/{init.sql, seed.sql, demo_queries.sql}
├── scripts/{build_index.py, smoke_test.py}
├── tests/
├── docs/
├── docker-compose.yml
├── requirements.txt
├── .env.example
└── README.md

```

7.1 Git çalışma kuralı

- main branch her akşam çalışır durumda tutulur.
- Günlük görevler için kısa feature branch kullanılır.
- Her kritik modül tamamlandığında küçük ve açıklayıcı commit atılır.
- Teslim öncesi v1.0.0 etiketi oluşturulur.
- Model dosyaları, veritabanı volume'leri ve secrets repository'ye eklenmez.

8. Yedi günlük ayrıntılı uygulama planı

Tarih	Odak	Gün sonu çıktısı
23 Temmuz	Teknik risk azaltma	Foundry chat + embedding, PostgreSQL ve temel UI/API çalışır
24 Temmuz	Plan analiz çekirdeği	SQL doğrulama, EXPLAIN JSON, parser ve kurallar
25 Temmuz	Knowledge base ve retrieval	20+ chunk, local embeddings, top-3 retrieval
26 Temmuz	Kaynaklı rapor	Foundry generation, Pydantic, citation ve fallback
27 Temmuz	UI ve evaluation	Tek sayfa demo, 12 senaryo ve metrics
28 Temmuz	Hardening ve dokümantasyon	Testler, README, mimari ve temiz kurulum
29 Temmuz	Release ve sunum	Demo videosu, 6 slayt, final evaluation, v1.0.0
30 Temmuz	Teslim günü	Smoke test ve link kontrolü; yeni özellik yok

23 Temmuz / Gün 1 - Teknik riskleri ilk gün kapatma

Amaç: Foundry Local chat ve embedding çağrıları ile PostgreSQL EXPLAIN temelinin gerçekten çalıştığını doğrulamak.

Çalışma bloğu	Yapılacaklar
Sabah	Repository, Python ortamı, .env.example ve paket kurulumu. Foundry Local SDK başlatma.
Öğle	Chat modeli indirme/yükleme ve tek prompt testi. Embedding modeliyle tek ve batch embedding testi.
Akşam	PostgreSQL Docker, /health endpoint'i, Streamlit'in API'ye test çağırısı ve teknik spike notları.

Tahmini yoğunluk: 8-10 saat; en kritik gün.

Gün sonu kabul kriterleri

- Foundry Local chat cevabı alınır.
- Embedding vektörü üretilir.
- PostgreSQL container ve /health endpoint'i çalışır.
- Streamlit ekranı açılır.
- Embedding SDK sorunu varsa aynı gün fallback kararı yazılı hale getirilir.

24 Temmuz / Gün 2 - PostgreSQL demo ve plan analiz motoru

Amaç: Sorgu -> güvenlik kontrolü -> EXPLAIN JSON -> normalize plan -> rule evidence akışını tamamlamak.

Çalışma bloğu	Yapılacaklar
Sabah	customers, orders ve support_events tabloları; generate_series ile sentetik veri ve dört demo sorgusu.
Öğle	SQLGlot tabanlı validator, read-only kullanıcı ve 3 saniyelik statement_timeout.
Akşam	Recursive plan parser, dört başlangıç kuralı ve unit test fixture'ları.

Tahmini yoğunluk: 9-11 saat.

Gün sonu kabul kriterleri

- Dört demo sorgusu EXPLAIN JSON üretir.
- Plan tree recursive parse edilir.
- En az üç problem kategorisi doğru sinyal verir.
- Yasaklı SQL testleri reddedilir.

25 Temmuz / Gün 3 - Knowledge base ve local retrieval

Amaç: PostgreSQL performans bilgisini kısa, temiz ve citation üretmeye uygun bir yerel bilgi tabanına dönüştürmek.

Çalışma bloğu	Yapılacaklar
Sabah	EXPLAIN, index, join, statistics, sorting ve multicolumn index konularında 6 Markdown not.
Öğle	Heading-aware chunker, metadata şeması ve 20+ chunk üretimi.
Akşam	Foundry Local batch embedding, .npz index, JSON metadata ve top-3 cosine retrieval testleri.

Tahmini yoğunluk: 8-10 saat.

Gün sonu kabul kriterleri

- En az 20 chunk üretilir.
- Embedding index diske kaydedilir.
- Dört demo probleminde beklenen kaynak top-3 içinde bulunur.
- Retrieval testleri geçer.

26 Temmuz / Gün 4 - Foundry Local ile kaynaklı analiz raporu

Amaç: Rule evidence ve retrieved context'i doğrulanan, citation-grounded bir kullanıcı raporuna dönüştürmek.

Çalışma bloğu	Yapılacaklar
Sabah	System prompt, user prompt, Pydantic şemaları ve provider adapter.
Öğle	Citation integrity, JSON parse, tek repair denemesi ve insufficient-context davranışı.
Akşam	POST /api/v1/analyses entegrasyonu ve LLM kapalıyken deterministik fallback.

Tahmini yoğunluk: 9-11 saat.

Gün sonu kabul kriterleri

- API kaynaklı rapor döndürür.
- Pydantic doğrulaması ve citation kontrolü çalışır.
- No-answer senaryosu çalışır.
- Foundry Local hatasında güvenli fallback döner.

27 Temmuz / Gün 5 - Streamlit arayüzü ve evaluation

Amaç: Sistemi sunulabilir hale getirmek ve başarı düzeyini ölçen otomatik değerlendirme üretmek.

Çalışma bloğu	Yapılacaklar
Sabah	Tek sayfa UI: senaryo, SQL editörü, Analyze, evidence, recommendation, citations, raw JSON.
Öğle	12 senaryoluk dataset: canlı query + kayıtlı plan fixture karışımı.
Akşam	Diagnosis accuracy, Hit@3, schema-valid, citation-valid, no-answer ve latency ölçümü.

Tahmini yoğunluk: 9-11 saat.

Gün sonu kabul kriterleri

- UI uçtan uca çalışır.
- 12 evaluation senaryosu vardır.
- Tek komutla results.json üretilir.
- Metrikler Markdown veya UI'da gösterilir.

28 Temmuz / Gün 6 - Test, hardening ve dokümantasyon

Amaç: Başka bir bilgisayarda kurulabilir ve teknik görüşmede savunulabilir bir proje oluşturmak.

Çalışma bloğu	Yapılacaklar
Sabah	SQL güvenliği, parser, rule engine, retrieval, API ve smoke testlerinin tamamlanması.

Çalışma bloğu	Yapılacaklar
Öğle	README: problem, mimari, kurulum, demo, evaluation, güvenlik, sınırlamalar.
Akşam	Temiz ortam setup, ekran görüntüleri, architecture.md ve bilinen sorunlar.

Tahmini yoğunluk: 8-10 saat.

Gün sonu kabul kriterleri

- pytest tamamen geçer.
- .env.example ve requirements.txt günceldir.
- README ve mimari diyagram tamamlanır.
- Temiz kurulum ve smoke test doğrulanır.

29 Temmuz / Gün 7 - Sunum, video ve release

Amaç: Teknik çıktıyı tekrar üretilebilir bir sunum ve teslim paketine dönüştürmek.

Çalışma bloğu	Yapılacaklar
Sabah	Final evaluation, README metrikleri, repository temizliği ve security scan.
Öğle	6 slaytlık sunum ve 5 dakikalık demo senaryosu provası.
Akşam	Demo videosu, sunum PDF'i, final smoke test ve v1.0.0 tag.

Tahmini yoğunluk: 8-10 saat.

Gün sonu kabul kriterleri

- Repo public veya incelemeye açıktır.
- Final evaluation sonucu kaydedilir.
- Sunum PDF'i ve demo videosu hazırdır.
- v1.0.0 release/tag oluşturulur.

30 Temmuz - Teslim günü protokolü

- Yeni özellik yazılmaz.
- Temiz ortamda kurulum ve model erişimi doğrulanır.
- Docker Compose sıfırdan başlatılır.
- Smoke test ve en az iki demo senaryosu çalıştırılır.
- README, video, sunum ve repository linkleri kontrol edilir.
- Yalnızca blocker hata veya metin/görsel düzeltmesi yapılır.

9. Test ve evaluation stratejisi

9.1 Test katmanları

Katman	Kapsam
Unit	SQL validator, recursive parser, rule thresholds, citation validator
Retrieval	Beklenen document_id'nin top-3 içinde bulunması
API	Başarılı analiz, invalid SQL, timeout, Foundry failure ve fallback

Katman	Kapsam
Smoke	PostgreSQL + API + UI üzerinde tek uçtan uca demo
Evaluation	12 etiketli query-plan senaryosu ve otomatik metrik raporu

9.2 Evaluation dataset dağılımı

Kategori	Adet	Kaynak
potential_missing_index	3	Canlı + fixture
expensive_nested_loop	2	Canlı veya fixture
disk_based_sort	2	Fixture ağırlıklı
cardinality_misestimation	2	Fixture
no_clear_issue	1	Canlı sağlıklı sorgu
insufficient_context / edge	2	Fixture

9.3 Raporlanacak metrikler

Metrik	Hesaplama
Rule diagnosis accuracy	Beklenen kategori ile rule engine çıktısının eşleşmesi
Retrieval Hit@3	Beklenen kaynak dokümanın ilk 3 retrieval sonucu içinde olması
Schema-valid response rate	Pydantic validation geçen model çıktısı / toplam
Citation-valid response rate	Retrieved set içinden geçerli citation kullanan çıktı / toplam
No-answer accuracy	Yetersiz bağlam senaryolarında kesin öneriden kaçınma
Latency	Ortalama ve p95 uçtan uca analiz süresi

Metrik dürüstlüğü

CV ve sunumda yalnızca final evaluation runner tarafından üretilen gerçek sonuçlar kullanılacaktır. Hedeflenen değere ulaşılamaması başarısızlık değildir; sınırlamaların açıkça raporlanması mühendislik kalitesinin parçasıdır.

10. Güvenlik, etik kullanım ve sınırlamalar

Risk	Kontrol
Zararlı SQL	AST doğrulama, allowlist yaklaşımı ve read-only DB kullanıcısı
Uzun sorgu	statement_timeout ve yalnızca demo veritabanı
LLM hallucination	Deterministik evidence, citation zorunluluğu, no-answer ve fallback
Otomatik değişiklik	recommendation_sql yalnızca gösterilir; asla otomatik uygulanmaz

Risk	Kontrol
Veri gizliliği	Yerel sentetik data ve Foundry Local on-device inference
Yanlış güven	UI ve README'de DBA incelemesinin yerini almadığı açıkça belirtilir

10.1 Bilinen sınırlamalar

- Yalnızca PostgreSQL ve kontrollü SELECT sorguları desteklenir.
- Kural eşikleri sentetik dataset üzerinde kalibre edilir; her production iş yüküne genellenemez.
- Execution plan; veri dağılımı, istatistikler, PostgreSQL sürümü ve donanıma göre değişebilir.
- Index önerisi yazma maliyeti, disk kullanımı ve write amplification açısından otomatik olarak değerlendirilmez.
- Küçük yerel model karmaşık planları açıklarken dilsel hata yapabilir; teknik gerçek rule evidence'dır.
- Proje eğitim ve portföy amaçlıdır; production DBA karar mekanizması değildir.

11. Risk kaydı ve önleyici aksiyonlar

Risk	Olasılık/Etki	Önlem
Foundry Local kurulumu gecikir	Yüksek/Yüksek	Gün 1 spike; chat korunur, embedding için belgelenmiş fallback
Embedding modeli donanımda çalışmaz	Orta/Yüksek	Daha küçük model alias veya sentence-transformers fallback; CV metni güncellenir
Canlı sorgular yavaşlık üretmez	Orta/Orta	Fixture planlar ve daha büyük generate_series dataset
LLM JSON'u bozuyor	Yüksek/Orta	Pydantic, tek repair denemesi ve deterministik fallback
Kapsam büyür	Yüksek/Yüksek	P0 listesi ve 30 Temmuz feature freeze
UI zaman kaybettirir	Orta/Orta	Tek sayfa Streamlit; görsel mükemmellik yerine işlev
Evaluation geç kalır	Orta/Yüksek	Dataset Gün 5'te zorunlu; kalan senaryolar fixture olabilir
Teslim günü ortam bozulur	Orta/Yüksek	Gün 28 temiz kurulum, Gün 29 release, offline demo video

12. Sunum, demo ve teslim paketi

12.1 Altı slaytlık sunum

Slayt	Başlık	İçerik
1	Problem ve motivasyon	Execution plan okumadaki zorluk ve local AI ihtiyacı
2	Ürün akışı	SQL -> EXPLAIN -> rules -> RAG -> Foundry Local -> report
3	Teknik mimari	FastAPI, PostgreSQL, local embeddings, Streamlit

Slayt	Başlık	İçerik
4	Güvenilirlik	Deterministik evidence, citation, no-answer, fallback
5	Evaluation	12 senaryo ve gerçek metrikler
6	Demo sonucu ve gelecek çalışmalar	Önce/sonra, sınırlamalar, tool-calling v2

12.2 Beş dakikalık demo akışı

Süre	Demo adımı
00:00-00:30	Problem ve local çalışma gerekçesi
00:30-01:10	Yavaş sorgu ve ilk çalışma süresi
01:10-02:05	Analizi başlatma ve plan evidence gösterimi
02:05-03:10	RAG kaynakları, açıklama ve öneri
03:10-04:00	Önerilen indexi demo DB'de manuel uygulama
04:00-04:35	Sorguyu yeniden çalıştırıp önce/sonra karşılaştırma
04:35-05:00	Evaluation, güvenlik ve sınırlamalar

12.3 Teslim paketi

- GitHub repository ve v1.0.0 tag
- İngilizce README ve kurulum talimatları
- Mimari diyagram ve architecture.md
- evaluation/scenarios.jsonl ve final results.json
- Ekran görüntüleri
- Altı slaytlık teknik sunum PDF'i
- Yaklaşık beş dakikalık demo videosu
- Bilinen sınırlamalar ve gelecekteki geliştirmeler listesi

13. 30 Temmuz teslim kontrol listesi

Alan	Kontrol maddesi
Teknik	Foundry Local chat modeli yerel cevap üretiyor
Teknik	Yerel embedding ve semantic retrieval çalışıyor
Teknik	PostgreSQL EXPLAIN JSON alınabiliyor
Teknik	En az dört rule engine kategorisi var
Güvenlik	Yalnızca read-only SELECT kabul ediliyor
Güvenlik	Recommendation SQL otomatik uygulanmıyor
Kalite	pytest ve smoke test geçiyor
Evaluation	12 senaryo ve final results.json mevcut
Dokümantasyon	README ve mimari açıklama tamamlandı

Alan	Kontrol maddesi
Sunum	Sunum PDF'i ve demo videosu hazır
Repository	Secrets yok, kurulum temiz ortamda doğrulandı
Release	v1.0.0 tag oluşturuldu ve teslim linkleri çalışıyor

14. CV ve portföy kullanımı

Aşağıdaki metin yalnızca proje gerçekten tamamlanıp sunulduktan ve final evaluation sonuçları doğrulandıktan sonra kullanılmalıdır.

Microsoft Türkiye | AI Innovators Summer Program Participant
Summer 2026

- Developed and presented QueryPilot Local, an offline PostgreSQL query-plan assistant using Microsoft Foundry Local, local embeddings, RAG, and structured EXPLAIN JSON analysis.
- Built a safety-first pipeline combining read-only SQL validation, deterministic plan diagnostics, citation-grounded generation, and regression evaluation across curated query-plan scenarios.
- Published the implementation, architecture, evaluation results, and technical documentation on GitHub and delivered a final technical presentation.

Final metrikler elde edildiğinde ikinci veya üçüncü maddeye diagnosis accuracy, Hit@3, citation-valid rate ve latency değerleri eklenebilir. Ölçülmemiş sayı kullanılmamalıdır.

15. Kaynakça

- [1] Microsoft Learn - Integrate with inference SDKs (Foundry Local) - [resmî bağlantı](#)
- [2] Microsoft Learn - Generate text embeddings (Foundry Local) - [resmî bağlantı](#)
- [3] PostgreSQL Documentation - Using EXPLAIN - [resmî bağlantı](#)
- [4] PostgreSQL Documentation - EXPLAIN command - [resmî bağlantı](#)
- [5] Kullanıcı tarafından sağlanan: One-Month Project Plan - Local RAG AI Assistant with Microsoft Foundry Local

Son karar

Teslim stratejisi

Projenin başarısı yeni özellik sayısıyla değil; güvenli çalışan ana akış, kaynaklı açıklama, otomatik evaluation, temiz kurulum ve savunulabilir teknik kararlarla ölçülecektir. 30 Temmuz'a kadar kapsam büyütülmeyecektir.